

ASYNCHRONOUS FIFO

Verification Results, Complete Signal Reference, and Next-Test Roadmap

CURRENT DESIGN	VERIFICATION	RESULT
8-bit data 8 entries 4-bit pointers	5 directed self-checking tests Two unrelated clocks	PASS 0 reported errors Finish: 1590 ns

What this guide gives you

- The exact five test cases currently implemented in `async_fifo_tb.v`, including stimulus, checks, and why each test matters.
- A complete revision list of the FIFO ports, parameters, storage, binary and Gray pointers, synchronizer stages, and next-state signals.
- A prioritized list of future plain-Verilog verification work, followed by stronger SystemVerilog, CDC, and formal options.

Verification evidence

The saved project record reports Vivado 2024.1 XSim PASS at 1590 ns. On 12 July 2026, the same RTL and testbench were independently compiled with Icarus Verilog and again printed **ASYNC_FIFO_TESTS: PASS** and **All 5 basic tests passed**.

Revision anchor

Binary pointers address memory and count wraparound. Gray pointers cross clock domains because only one Gray bit changes per adjacent count. Each crossing uses two flip-flops. The write side decides **full** from the next write Gray pointer and synchronized read Gray pointer; the read side decides **empty** from the next read Gray pointer and synchronized write Gray pointer.

1. Tests currently implemented

The testbench is self-checking: every mismatch increments **errors**. A clean run reaches the final PASS message; any accumulated error reaches FAIL and **\$fatal**.

ID	Purpose	Stimulus and checks	Expected behavior	Status
T01	Reset and empty-read protection	Assert both resets; verify empty=1, full=0, data_out=00. Then pulse rd_en while empty and save/compare dut.rd_bin.	Reset outputs are correct. Empty read is rejected and the read pointer does not move.	PASS
T02	One write, one read	Write A5, wait until the read domain sees non-empty, read once, and compare data_out with A5.	A5 returns unchanged and empty returns to 1 after the accepted read.	PASS
T03	FIFO ordering	Write 10, 11, 12, 13; then read four times with expected values in the same order.	Every value emerges in first-in, first-out order with no loss or reordering.	PASS
T04	Full and overflow protection	Write 80 through 87 into the 8-entry FIFO. Check full=1. Attempt EE while full and compare dut.wr_bin before/after. Drain all eight words.	Full asserts; EE is rejected; write pointer does not move; original eight values remain intact.	PASS
T05	Pointer wraparound	Run two complete fill/drain rounds: 00-07, then 08-0F. At the end compare dut.wr_bin and dut.rd_bin with zero.	Data survives both rounds and both 4-bit pointers wrap back to 0000.	PASS

Clock and task behavior

Item	Current setting	Why it matters
wr_clk	10 ns period (100 MHz)	Writes occur only on write-clock rising edges.
rd_clk	14 ns period (about 71.43 MHz), starting 2 ns late	Different frequency and phase exercise real clock-domain separation.
write_byte	Drive at falling edges; keep wr_en high across one rising edge	Creates one unambiguous write opportunity.
read_byte	Wait for !empty; enable across a read rising edge; sample 1 ns later	Allows synchronized visibility and checks registered data_out.
reset_fifo	Assert both resets for 20 ns; release near separate falling edges	Starts both domains from a known state without changing stimulus at an active edge.

What these five tests establish

They establish basic reset state, underflow blocking, single-transfer correctness, FIFO order, full detection, overflow blocking, draining, and binary-pointer wraparound for the default 8 x 8 configuration.

2. External signals and configuration

The FIFO has two independent timing domains. A signal should be interpreted in the domain named in the first column; do not assume that **full** and **empty** change together.

Domain	Signal	Direction / width	Use
Write	wr_clk	input, 1 bit	Write-domain clock. Accepted writes, write pointer updates, full updates, and read-pointer synchronization occur on its rising edge.
Write	wr_reset	input, 1 bit	Active-high asynchronous reset for write-domain state: wr_bin, wr_gray, full, and both read-Gray synchronizer stages.
Write	wr_en	input, 1 bit	Write request. A write occurs only when wr_en=1 and full=0.
Write	data_in	input, DATA_WIDTH	Word presented for storage. It is written into memory at the current write address when a write is accepted.
Write	full	output reg, 1 bit	Write-domain protection flag. When 1, no free entry is safely visible to the writer and further writes are blocked.
Read	rd_clk	input, 1 bit	Read-domain clock. Accepted reads, read pointer updates, empty updates, data_out updates, and write-pointer synchronization occur on its rising edge.
Read	rd_reset	input, 1 bit	Active-high asynchronous reset for read-domain state: rd_bin, rd_gray, data_out, empty, and both write-Gray synchronizer stages.
Read	rd_en	input, 1 bit	Read request. A read occurs only when rd_en=1 and empty=0.
Read	data_out	output reg, DATA_WIDTH	Registered read data. It changes only on an accepted read; reset drives it to zero.
Read	empty	output reg, 1 bit	Read-domain protection flag. When 1, no unread entry is safely visible to the reader and further reads are blocked.

Parameters and derived constant

Name	Default	Meaning
DATA_WIDTH	8	Number of bits stored in each FIFO word. Controls data_in, data_out, and every memory entry.
ADDRESS_WIDTH	3	Number of RAM address bits. Depth is 2 raised to this value. Binary and Gray pointers use one extra wrap bit.
DEPTH	$1 \ll ADDRESS_WIDTH = 8$	Total number of entries in the current default FIFO.

Default-width rule

With ADDRESS_WIDTH=3, the memory address is 3 bits, but each read/write pointer is 4 bits. The low 3 bits select one of 8 locations; the extra bit distinguishes different laps around the circular buffer.

3. Internal signal reference

This table covers every state element and combinational wire declared inside **async_fifo**. Widths below are parameterized; for the current configuration, ADDRESS_WIDTH+1 equals 4 bits.

Group	Signal	Width / type	Use
Storage	memory[0:DEPTH-1]	DEPTH x DATA_WIDTH reg array	Holds FIFO words. Write side updates one entry on an accepted write; read side fetches one entry on an accepted read.
Pointer	wr_bin	ADDRESS_WIDTH+1 reg	Current binary write pointer. Low bits address memory; extra bit tracks wrap. Updates only in wr_clk domain.
Pointer	rd_bin	ADDRESS_WIDTH+1 reg	Current binary read pointer. Low bits address memory; extra bit tracks wrap. Updates only in rd_clk domain.
CDC pointer	wr_gray	ADDRESS_WIDTH+1 reg	Gray-coded version of wr_bin. This is the write pointer representation sent toward the read domain.
CDC pointer	rd_gray	ADDRESS_WIDTH+1 reg	Gray-coded version of rd_bin. This is the read pointer representation sent toward the write domain.
Synchronizer	rd_gray_sync1	ADDRESS_WIDTH+1 reg	First write-clocked sampling stage for rd_gray. It may absorb metastability and is not used for the full decision.
Synchronizer	rd_gray_sync2	ADDRESS_WIDTH+1 reg	Second write-clocked stage. Stable read-pointer view used by write-domain full logic.
Synchronizer	wr_gray_sync1	ADDRESS_WIDTH+1 reg	First read-clocked sampling stage for wr_gray. It may absorb metastability and is not used for the empty decision.
Synchronizer	wr_gray_sync2	ADDRESS_WIDTH+1 reg	Second read-clocked stage. Stable write-pointer view used by read-domain empty logic.
Gate	write_allowed	wire, 1 bit	wr_en && !full. It is 1 exactly when the next write edge may consume an entry.
Gate	read_allowed	wire, 1 bit	rd_en && !empty. It is 1 exactly when the next read edge may consume an unread word.
Next state	wr_bin_next	ADDRESS_WIDTH+1 wire	wr_bin plus 1 when write_allowed, otherwise unchanged.
Next state	rd_bin_next	ADDRESS_WIDTH+1 wire	rd_bin plus 1 when read_allowed, otherwise unchanged.
Next state	wr_gray_next	ADDRESS_WIDTH+1 wire	Gray conversion of wr_bin_next: (wr_bin_next >> 1) XOR wr_bin_next.
Next state	rd_gray_next	ADDRESS_WIDTH+1 wire	Gray conversion of rd_bin_next: (rd_bin_next >> 1) XOR rd_bin_next.
Flag next	full_next	wire, 1 bit	Compares wr_gray_next with synchronized read Gray pointer after inverting its top two bits. Registered into full on wr_clk.
Flag next	empty_next	wire, 1 bit	Compares rd_gray_next with wr_gray_sync2. Registered into empty on rd_clk.

Why the synchronizer registers are marked ASYNC_REG

The ASYNC_REG=TRUE attributes tell FPGA implementation tools that these registers form clock-domain synchronizer chains. This helps preserve their structure and encourages placement that improves metastability settling time.

4. How the signals work together

Accepted operations and blocked operations

Case	Condition	Result
Accepted write	wr_en=1 and full=0 at wr_clk rising edge	Store data_in at memory[wr_bin address]; advance wr_bin/wr_gray; recompute/register full.
Blocked write	wr_en=1 and full=1	write_allowed=0; memory and write pointer remain unchanged. This prevents overwriting unread data.
Accepted read	rd_en=1 and empty=0 at rd_clk rising edge	Load data_out from memory[rd_bin address]; advance rd_bin/rd_gray; recompute/register empty.
Blocked read	rd_en=1 and empty=1	read_allowed=0; data_out and read pointer remain unchanged. This prevents reading invalid data.

Flag meaning across asynchronous domains

- **empty** is intentionally conservative. A new write must cross two read-clocked synchronizer stages before the reader can safely see it.
- **full** is also conservative. A completed read must cross two write-clocked synchronizer stages before the writer can safely reclaim that location.
- Therefore, flag deassertion can lag the physical operation by a few destination-clock edges. This is safe behavior, not a functional error.

```
full_next = (wr_gray_next == {invert top two bits of rd_gray_sync2, keep remaining bits})
empty_next = (rd_gray_next == wr_gray_sync2)
```

Testbench-only signals

Name	Type	Verification use
wr_clk, rd_clk	reg	Generated clocks that drive the DUT.
wr_reset, rd_reset	reg	Testbench-controlled asynchronous reset inputs.
wr_en, rd_en	reg	Testbench request controls.
data_in	reg [7:0]	Stimulus byte driven into the FIFO.
full, empty	wire	Observed DUT protection flags.
data_out	wire [7:0]	Observed DUT read data.
errors	integer	Self-check failure counter; must finish at zero.
i, round	integer	Loop variables for sequence and wraparound tests.
saved_pointer	reg [3:0]	Captures a pointer before a forbidden operation so movement can be detected.

Fast waveform reading order

First identify wr_clk versus rd_clk. Then check reset, enable, and full/empty. Next confirm the corresponding binary pointer moves only on an accepted edge. Finally check memory data through data_in/data_out and use Gray/synchronized pointers to explain delayed flag visibility.

5. Future Verilog verification roadmap

The present suite is a strong beginner baseline, but it is directed and short. The highest-value next step is a scoreboard-driven stress test that runs reads and writes concurrently under several clock relationships.

Priority	Test	How to add it	What it can reveal
P1	Concurrent mixed traffic	Run wr_en and rd_en independently for hundreds of cycles while a reference queue predicts every accepted read.	Find data loss, duplication, or reordering that sequential fill-then-drain tests may miss.
P1	Clock-ratio matrix	Repeat with write much faster, read much faster, nearly equal periods, and different phase offsets such as 10/14, 6/17, 11/13 ns.	Stress both pointer crossings and flag latency in both directions.
P1	Boundary transitions	Fill to 7, write the 8th word, attempt writes while full, read one, and verify full clears only after synchronized visibility. Mirror this around empty.	Verify exact behavior at occupancy 0, 1, DEPTH-1, and DEPTH.
P1	Reset during traffic	Assert wr_reset, rd_reset, and both resets at varied times while requests are active. First define the intended independent-reset policy.	Expose stale pointers, phantom data, and mismatched-domain recovery assumptions.
P2	Long randomized regression	Use a deterministic pseudo-random generator or \$random seed to vary enables, data, idle gaps, and bursts. Log the seed.	Discover interleavings not covered by five directed sequences and make failures reproducible.
P2	Repeated illegal requests	Hold wr_en high for many clocks while full and rd_en high while empty, not just one attempted operation.	Prove pointers and stored data remain stable throughout prolonged overflow/underflow pressure.
P2	Parameter sweep	Compile/run DATA_WIDTH values such as 1, 8, 16, 32 and ADDRESS_WIDTH values such as 2, 3, 4.	Catch hard-coded widths, slice mistakes, and full-comparator corner cases.
P2	Many wraparounds	Run dozens or hundreds of full pointer laps with mixed bursts instead of exactly two clean rounds.	Stress the extra pointer bit and repeated address reuse.
P2	Reset release timing	Release each reset just before/after its own clock edge and at unrelated times to the other clock.	Check deterministic domain startup and flag convergence.
P3	Unknown-state checks	Monitor important outputs and pointers for X/Z after reset release; intentionally vary initialization in simulation.	Reveal dependence on simulator initialization rather than explicit reset behavior.
P3	Assertions and coverage	Move to SystemVerilog assertions plus covergroups after the plain-Verilog scoreboard is stable.	Automate invariants and measure whether boundaries, wraps, and simultaneous activity were actually exercised.
P3	CDC and formal checks	Run a structural CDC report and formal properties for no overflow, no underflow, pointer stability, and ordering.	Address CDC topology and exhaustive control-state behavior beyond event simulation.

6. Recommended next testbench architecture

You can implement the next stage in plain Verilog before adopting classes or UVM. Keep the current five directed tests as a smoke suite, then add one separate stress testbench.

Reference-model recipe

- Create a model array plus model write/read indices and a model_count.
- Whenever wr_en && !full is true at a write rising edge, push data_in into the model.
- Whenever rd_en && !empty is true at a read rising edge, compare data_out with the oldest model entry, then pop it.
- Never update the model for blocked writes or blocked reads.
- Count accepted writes, accepted reads, blocked operations, full/empty transitions, and pointer wraps.
- Stop after a fixed accepted-operation count and require zero scoreboard errors.

Core invariants to monitor

Invariant	Required behavior
No overflow movement	If wr_en && full at a write edge, wr_bin and wr_gray must remain unchanged.
No underflow movement	If rd_en && empty at a read edge, rd_bin and rd_gray must remain unchanged and data_out must not be replaced.
Gray single-step	Whenever an accepted operation advances a Gray pointer, XOR(old_gray, new_gray) must contain exactly one 1 bit.
Data ordering	Every accepted read must equal the oldest not-yet-read accepted write in the reference model.
Reset state	After reset is observed in a domain, that domain's pointer and synchronizer registers are zero; full=0 on write reset and empty=1/data_out=0 on read reset.
Known values	After reset release and settling, full, empty, pointers, synchronizer stages, and checked output data must never be X or Z.

Revision checklist

Question	Answer you should remember
Why two clocks?	Producer and consumer operate independently; there is no fixed phase relationship.
Why binary pointers?	Efficient memory addressing and arithmetic.
Why Gray pointers?	Only one bit changes per adjacent count, making synchronized comparisons safer.
Why two synchronizer stages?	The second stage receives extra settling time and is the stable value used by destination logic.
Why next-pointer flag logic?	The registered flag describes whether the operation leading to the next state reaches a boundary.
What simulation cannot prove?	Ordinary RTL simulation does not model analog metastability probability or guarantee physical CDC implementation quality.

Best immediate next step

Add a plain-Verilog concurrent random testbench with a reference queue, then run a clock-ratio matrix and reset-during-traffic cases. This gives the largest verification gain while keeping the environment understandable.

7. Which signals were added - and why

Important interface clarification

No new top-level FIFO pin was silently added. The current interface is the same functional interface under clearer names. The additional items are internal helper wires, registered flag state, implementation attributes, or testbench-only variables. Internal and testbench-only signals do not create extra FPGA package pins.

Ownership legend

Label	Meaning
Retained / renamed	The same design idea existed in your original RTL; only the name or coding style changed.
Added helper	A new internal name was introduced to make acceptance, next state, or registered flag behavior explicit.
Verification only	Created only inside <code>async_fifo_tb.v</code> . It exists in simulation and is not synthesized as FIFO hardware.
Tool metadata	Not a logic signal. It is an instruction to the FPGA implementation tool.

Original-to-current name map

Original form	Current form	Classification	Purpose
wren / rden	wr_en / rd_en	Retained / renamed	Write and read requests.
din / dout	data_in / data_out	Retained / renamed	Input word and registered output word.
wrclk / rdclk	wr_clk / rd_clk	Retained / renamed	Independent write/read clocks.
wrrst / rdrst	wr_reset / rd_reset	Retained / renamed	Per-domain asynchronous resets.
wrptrbin / rdptrbin	wr_bin / rd_bin	Retained / renamed	Binary address-plus-wrap pointers.
wrptrgray / rdptrgray	wr_gray / rd_gray	Retained / renamed	Gray pointers that cross domains.
rdptrgray1 / 2	rd_gray_sync1 / 2	Retained / renamed	Read pointer synchronization into write domain.
wrptrgray1 / 2	wr_gray_sync1 / 2	Retained / renamed	Write pointer synchronization into read domain.
wrptrbin_next / rdptrbin_next	wr_bin_next / rd_bin_next	Retained / restyled	Original next-state regs are now direct combinational wires.
wrptrgray_next / rdptrgray_next	wr_gray_next / rd_gray_next	Retained / restyled	Original Gray next-state signals are now direct wires.
Repeated wren && !full conditions	write_allowed	Added helper	One named decision for an accepted write.
Repeated rden && !empty conditions	read_allowed	Added helper	One named decision for an accepted read.
Direct combinational full output	full_next plus registered full	Added helper / state	Separates the predicted next flag from the stable write-domain output.
Direct combinational empty output	empty_next plus registered empty	Added helper / state	Separates the predicted next flag from the stable read-domain output.

Why rename signals at all?

Names such as `wr_bin`, `rd_gray_sync2`, and `write_allowed` expose both direction and role at a glance. The functionality is not hidden behind shorter names, which makes waveforms, reviews, and failure messages easier to read.

8. Deep explanation of the added hardware helpers

write_allowed and read_allowed

These are one-bit combinational acceptance signals. They are not clocks, registers, or external handshakes. They simply name the exact condition under which a request is legal.

```
write_allowed = wr_en && !full
```

When it is 1, the next write rising edge stores data and advances the write pointer. When it is 0, both the write pointer and memory write are suppressed. Naming this decision once guarantees that memory movement and pointer movement use the same rule. Without the helper, the condition must be repeated in several places; a later edit could accidentally make one copy different from another.

```
read_allowed = rd_en && !empty
```

When it is 1, the next read rising edge loads data_out and advances the read pointer. When it is 0, the read pointer and data_out remain unchanged. This is the central underflow-protection decision.

wr_bin_next and rd_bin_next

Each next-binary signal answers: *what should this domain's pointer become at its next active edge?* If an operation is accepted, the answer is current pointer plus one. Otherwise, it is the current pointer. This explicit next-state form lets pointer conversion and flag prediction use the same future state that will be registered.

```
wr_bin_next = wr_bin + write_allowed
rd_bin_next = rd_bin + read_allowed
```

Because write_allowed/read_allowed are exactly 0 or 1, adding the signal means hold or increment. The extra pointer MSB is included in the addition, so wraparound occurs naturally.

wr_gray_next and rd_gray_next

These wires convert the predicted binary pointers into Gray code before the clock edge. They are needed because full/empty must be predicted from the state that the accepted operation will create, not only from the state before that operation.

```
gray_next = (binary_next >> 1) XOR binary_next
```

Adjacent binary counts can change several bits at once, but adjacent reflected Gray counts change one bit. That one-bit transition is why Gray pointers, rather than binary pointers, are sent through the two-flop clock-domain synchronizers.

full_next

full_next predicts whether the write pointer after the possible write would be exactly one FIFO depth ahead of the synchronized read pointer. For a power-of-two asynchronous FIFO, this condition is recognized in Gray code by inverting the top two synchronized read-pointer bits and keeping the lower bits unchanged.

```
full_next = (wr_gray_next == {~rd_gray_sync2[top:top-1], rd_gray_sync2[remaining]})
```

For the current 8-entry FIFO, suppose rd_gray_sync2 is 0000. One full-buffer distance ahead is binary 1000, whose Gray value is 1100. Inverting the top two bits of 0000 produces 1100, so the comparison correctly predicts full.

9. Added flag state, synchronizer metadata, and test signals

empty_next and registered empty

empty_next predicts whether the read pointer after a possible read equals the synchronized write pointer. Equality means that no unread item remains from the reader's safe, synchronized point of view.

```
empty_next = (rd_gray_next == wr_gray_sync2)
```

The current RTL stores empty in a read-clocked register. Therefore, empty changes only on rd_clk or rd_reset. Likewise, full is stored in a write-clocked register and changes only on wr_clk or wr_reset. This makes each public status output synchronous to the domain that consumes it and avoids a long combinational output path from synchronized pointer comparison to user logic.

Why full and empty may clear later

A read that frees space is not immediately visible in the write domain. rd_gray must pass through rd_gray_sync1 and rd_gray_sync2 before full_next can react. A write that adds data follows the mirror path through wr_gray_sync1 and wr_gray_sync2 before empty_next can react. This conservative delay protects data; it is not a missing transaction.

ASYNC_REG attributes

The ASYNC_REG=TRUE text attached to the four synchronizer registers is tool metadata, not another signal. It tells Vivado that the registers belong to CDC synchronizer chains so the tool can preserve and place them appropriately. Simulation logic does not change because of this attribute.

Testbench-only signals added for verification

Signal / reference	Why it was added	Hardware impact
errors	Counts failed comparisons. Final PASS requires errors=0.	Simulation only
i	Loop index for writing/reading ordered test patterns.	Simulation only
round	Counts the two full fill/drain wraparound rounds.	Simulation only
saved_pointer	Stores wr_bin or rd_bin before a forbidden operation, allowing the test to prove that the pointer did not move.	Simulation only
dut.wr_bin / dut.rd_bin references	Hierarchical observation of internal DUT state for directed checks. These references do not create ports or modify hardware.	Simulation only

Worked trace: blocked empty read

Step	Signal reasoning
1	empty=1 and rd_en=1, so read_allowed=0.
2	rd_bin_next equals rd_bin; therefore rd_gray_next also equals rd_gray.
3	At rd_clk, rd_bin and rd_gray reload the unchanged values.
4	The memory is not read into data_out because read_allowed is 0.
5	saved_pointer confirms that the forbidden read caused no pointer movement.

10. More worked signal traces

First write into an empty FIFO

Step	Signal reasoning
1	wr_en=1 and full=0, so write_allowed=1.
2	If wr_bin=0000, wr_bin_next becomes 0001 and wr_gray_next becomes 0001.
3	At wr_clk, data_in is stored at memory address 000 and the new write pointers are registered.
4	wr_gray crosses wr_gray_sync1 and then wr_gray_sync2 on read-clock edges.
5	After that safe crossing, empty_next becomes 0 and empty deasserts on rd_clk.

Blocked write while full

Step	Signal reasoning
1	full=1 and wr_en=1, so write_allowed=0.
2	wr_bin_next equals wr_bin and wr_gray_next equals wr_gray.
3	The memory write condition is false, so data_in does not overwrite an unread entry.
4	At wr_clk, the write pointers reload unchanged values; saved_pointer proves there was no movement.
5	full stays asserted until a read pointer update safely crosses rd_gray_sync1 and rd_gray_sync2.

Final accepted read that makes the FIFO empty

Step	Signal reasoning
1	rd_en=1 and empty=0, so read_allowed=1.
2	rd_bin_next advances to the synchronized write position and rd_gray_next is calculated from it.
3	Because rd_gray_next equals wr_gray_sync2, empty_next becomes 1.
4	At rd_clk, the final word is loaded into data_out and empty is registered as 1.
5	A later read request is blocked because read_allowed becomes 0.

How to classify any unfamiliar waveform signal

Question	Interpretation
Is it a module port?	It is part of the external FIFO interface.
Is it assigned with assign and declared wire?	It is combinational helper logic, not stored state.
Is it assigned inside a clocked always block?	It is a register/state element in that clock domain.
Is it declared only in async_fifo_tb?	It is verification-only and does not become FIFO hardware.
Is it inside (* ... *)?	It is synthesis/tool metadata rather than a runtime logic signal.

Revision rule

Helper wires describe decisions and future values; synchronizer registers carry Gray pointers across domains; full and empty are domain-local state; testbench variables only observe and check. None of these helpers changes the FIFO's basic purpose or adds an external data/control pin.